

DATENBANKEN für Hacker

Eigenständiger Vortrag von CryptoJones

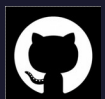
45 Minuten* • Kein Abschluss erforderlich



Aaron K. Clark

Masterstudent, Eastern University*

cryptojones@owasp.org



github.com/CryptoJones



CryptoJones@infosec.exchange

**Jeder Einbruch, von dem du je
gelesen hast, endet bei einer
DATENBANK***

Die Kronjuwelen



- ▶ Nicht deine Single-Page-App.
- ▶ Nicht deine Firewall.
- ▶ Nicht deine Microservices.
- ▶ DIE DATEN. Immer die Daten.

Was ist eine Datenbank?

Organisierter Speicher + eine Möglichkeit, Fragen zu stellen.

Das ist alles. Alles andere ist Detail.

- ▶ Aktenschrank = Speicher + deine Augen als Abfrage.
- ▶ Excel + Strg-F = Speicher + Suchen-und-Ersetzen als Abfrage.
- ▶ Eine echte DB ergänzt: Skalierung, Geschwindigkeit, Nebenläufigkeit, eine formale Sprache.

Die sechs Begriffe, die du brauchst

TABELLE

■ ein Tabellenblatt

ZEILE

■ ein Datensatz (ein Kunde, eine Warnung)

SPALTE

■ ein Attribut (E-Mail, IP, Summe)

ABFRAGE

■ die Frage, die du stellst

SCHEMA

■ die Struktur der Daten

INDEX

■ das Lesezeichen, das Suchvorgänge schnell macht

+ CRUD = Create • Read • Update • Delete

Geschichtstour — Fossilien, die du in der Produktion siehst

1960s	Flache Dateien	▶ Dateisystemrechte = einzige Zugriffskontrolle
Späte 1960er	Hierarchisch (IMS)	▶ Mainframe-Ära, physisches Bedrohungsmodell
1970s	Netzwerk / CODASYL	▶ läuft noch im Backoffice deiner Bank
1970	Codds Relationenmodell	▶ GRANT, Rollen — Zugriffskontrolle innerhalb der DB
1980s	Oracle, DB2, SQL Svr	▶ Datenbanken werden vernetzt → authN-Problem
1990s	Client-Server, OSS	▶ SQL-Injection wird zum Thema
2000s	NoSQL-Boom	▶ "warf das Fundament weg" — inkl. Sicherheit
2010s	Cloud / NewSQL	▶ geteilte Verantwortung, Fehlkonfigurationen durch Kunden
2020s	Vektor / serverless	▶ Datenvergiftung, Prompt-Injection

Zehn Datenbankfamilien

◆ 1. **Relational**
SQL • PostgreSQL, MySQL

◆ 3. **Key-Value**
Wörterbuch • Redis, DynamoDB

◆ 5. **Graph**
Knoten+Kanten • Neo4j, Neptune

◆ 7. **Time-Series**
Metriken • Influx, Prometheus

◆ 9. **Embedded/Edge**
in-app • SQLite, DuckDB

◆ 2. **Dokument**
JSON • MongoDB, Firestore

◆ 4. **Column-Family**
wide-column • Cassandra, HBase

◆ 6. **Suche**
Volltext • Elastic, OpenSearch

◆ 8. **Vektor**
Embeddings • Pinecone, pgvector

◆ 10. **Data Lake**
schema-on-read • S3+Athena, Iceberg

1

Relational / SQL

PRODUKTE

PostgreSQL • MySQL/MariaDB • SQL Server • Oracle

EINSETZEN WENN

Klare Beziehungen, Konsistenz erforderlich — Finanzen, regulierte Daten.

⚠ SICHERHEITSFALLE

SQL-Injection, wenn Entwickler Benutzereingaben in Abfragen verketteten.

2

Dokument-Stores

PRODUKTE

MongoDB • Couchbase • Firestore

EINSETZEN WENN

Natürlich verschachtelte Daten; die Struktur ändert sich mit der Zeit.

⚠ SICHERHEITSFALLE

NoSQL-Injection über Abfrageoperatoren (\$ne, \$gt, \$where).
Standardmäßig keine Authentifizierung.

3

Key-Value-Stores

PRODUKTE

Redis • Memcached • DynamoDB

EINSETZEN WENN

Caches, Session-Stores, Rate-Limiter, Bestenlisten.

SICHERHEITSFALLE

Standardmäßig auth-loses Redis + CONFIG SET → SSH-Schlüssel auf die Festplatte schreiben.

4

Column-Family

PRODUKTE

Apache Cassandra • HBase • ScyllaDB

EINSETZEN WENN

Massive Skalierung über viele Maschinen; keine komplexen Joins nötig.

SICHERHEITSFALLE

Cluster vertrauen einander über Gossip-Ports — Management-Netzwerk = Root.

5

Graphdatenbanken

PRODUKTE

Neo4j • Amazon Neptune • ArangoDB

EINSETZEN WENN

Soziale Netzwerke, Betrugserkennung, IAM-Analyse (z. B. BloodHound).

SICHERHEITSFALLE

Abfragesprachen wie Cypher sind ebenfalls injizierbar. NoSQLi ist nicht nur Mongo.

6

Suchmaschinen

PRODUKTE

Elasticsearch • OpenSearch • Solr

EINSETZEN WENN

Log-Suche, SIEM-Rückgrate, E-Commerce-Seitensuche.

⚠ SICHERHEITSFALLE

Historisch keine integrierte Authentifizierung — Milliarden Datensätze auf Shodan exponiert.



Time-Series

PRODUKTE

InfluxDB • TimescaleDB • Prometheus

EINSETZEN WENN

Metriken, Observability, IoT-Telemetrie.

SICHERHEITSFALLE

Üblicherweise innerhalb des Perimeters, oft ohne Authentifizierung.

8

Vektordatenbanken

PRODUKTE

Pinecone • Milvus • Weaviate • pgvector

EINSETZEN WENN

RAG, Empfehlungssysteme, semantische Suche.

SICHERHEITSFALLE

Datenvergiftung — schmuggle ein Dokument ein, das LLM zitiert es als Wahrheit.

9

Embedded / Edge

PRODUKTE

SQLite • DuckDB • LevelDB

EINSETZEN WENN

In-App-Speicher — Browser, Telefone, jedes moderne Flugzeug.

SICHERHEITSFALLE

Die DB ist eine Datei — der Angriff lautet 'stiehl die Datei'. Welt des mobilen Pentests.

10

Data Lakes / Lakehouses

PRODUKTE

S3 + Athena • Delta Lake • Iceberg • Snowflake

EINSETZEN WENN

Schema-on-read statt schema-on-write. Jetzt ablegen, später abfragen.

SICHERHEITSFALLE

Riesige S3-Pools + freizügiges IAM, keine erzwungene Verschlüsselung im Ruhezustand.

Cloud-DB-Dienste — Ein Überblick

AWS

- RDS, Aurora
- DynamoDB, DocumentDB
- Redshift, S3 + Athena
- ElastiCache
- Neptune (Graph)
- Timestream
- S3 (Objektspeicher)

GCP

- Cloud SQL, AlloyDB
- Firestore, Bigtable
- BigQuery
- Memorystore
- — (in Kürze)
- — (BYO)
- Cloud Storage

Azure

- Azure SQL DB
- Cosmos DB (Gestaltwandler)
- Synapse Analytics
- Azure Cache for Redis
- Cosmos DB Graph-Modus
- — (BYO)
- Blob Storage

Cloud-DB-Spickzettel

KATEGORIE	AWS	GCP	AZURE
Relational	RDS / Aurora	Cloud SQL / AlloyDB	Azure SQL DB
Dokument/NoSQL	DynamoDB / DocDB	Firestore / Bigtable	Cosmos DB
Warehouse	Redshift / Athena	BigQuery	Synapse
Cache	ElastiCache	Memorystore	Cache for Redis
Graph	Neptune	—	Cosmos DB Graph
Time-Series	Timestream	—	—
Objektspeicher	S3	Cloud Storage	Blob Storage

Modell der geteilten Verantwortung

✓ CLOUD-ANBIETER VERANTWORTET

- Hypervisor & Hardware
- DB-Engine-Patches
- Physische Sicherheit
- Zugrundeliegendes Netzwerk



DU VERANTWORTEST

- Konfiguration
- Zugriffsrichtlinien (IAM)
- Die Daten selbst
- Verschlüsselungsentscheidungen

Fast jeder Cloud-DB-Einbruch = Fehlkonfiguration durch den Kunden.

Gängige DB-Ports — Präge sie dir ein

3306	▶	MySQL / MariaDB
5432	▶	PostgreSQL
1433	▶	Microsoft SQL Server
27017	▶	MongoDB
6379	▶	Redis
9200	▶	Elasticsearch
9042	▶	Cassandra

Jeder davon im Internet exponiert = ein Befund. Jedes Mal*

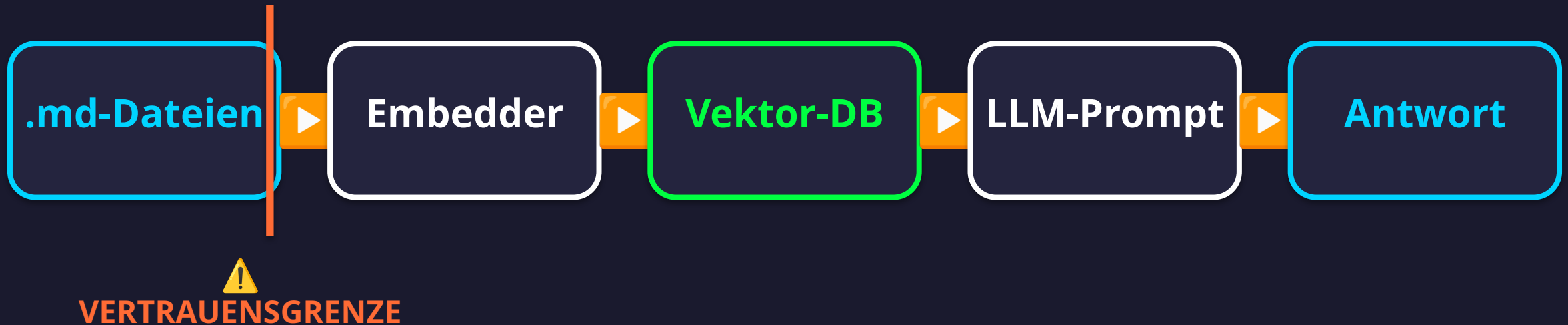
Deine Notizen sind jetzt eine Datenbank

Markdown-Dateien sind jetzt
die Datenbank, aus der ein LLM liest.

Obsidian-Vaults • Wiki-Exporte • Besprechungsnotizen • GitHub-Dokumente

Eingebettet → Vektor-DB → LLM-Kontext → Antwort an Benutzer

RAG-Pipeline — Wo ist die Vertrauensgrenze?



Wenn jeder diese .md-Dateien bearbeiten kann, kann jeder Inhalte platzieren, die die KI als maßgeblich zitiert.

Die Kernbotschaft

**Behandle Markdown wie eine
Datenbank.**

Denn für das LLM ist es eine.

- ▶ Zugriffskontrolle für den Dokumente-Ordner.
- ▶ Code-Review bei .md-Änderungen.
- ▶ Diff-Verfolgung im Wiki.

SICHERHEIT

Was 2026 tatsächlich angegriffen wird*

xkcd #327 — Bobby Tables

Schule: "Hallo, hier ist die Schule Ihres Sohnes. Wir haben ein paar Computerprobleme."

Mutter: "Oje — hat er etwas kaputt gemacht?"

Schule: "Haben Sie Ihren Sohn wirklich Robert');
DROP TABLE Students;-- genannt?"

<https://xkcd.com/327/>

Der Mechanismus: String-Verkettung in Abfragen.

Die Angriffsfläche hat sich
VERGRÖßSERT, nicht verkleinert.

SQL-Injection wurde behoben. Elf neue Kategorien traten an ihre Stelle.



NoSQL-Injection

MongoDB-JSON-Abfragen + rohe Benutzereingaben = SQLi neu geboren.

- Login-Umgehung: { username: 'alice', password: { \$ne: 'x' } }
- \$where erlaubt es, JavaScript direkt in die DB zu injizieren.
- Couchbase N1QL ist SQL-artig — klassische SQLi-Muster gelten.
- Prinzip: JEDER Interpreter kann injiziert werden, nicht nur SQL.

OWASP Top 10 2025: 'Dynamische Abfragen... direkt im Interpreter verwendet.'



Exponierte & fehlerkonfigurierte Datenbanken

Shodan und Censys sind voll von Datenbanken, die dort nicht sein sollten.

- MongoDB an 0.0.0.0 gebunden, ohne Authentifizierung.
- Elasticsearch-Cluster weit offen. Redis ohne Passwort.
- Postgres pg_hba.conf gesetzt auf 'host all all 0.0.0.0/0 trust'.
- Meow-Angriffe (2020): Bots löschten zum Spaß Tausende offener DBs.

Maßnahme: Finde heraus, was deine Organisation an öffentliche Schnittstellen gebunden hat. Heute.



Cloud-Fehlkonfigurationen

In der Cloud liegt der Fehler selten an der Engine. Es ist die Richtlinie drumherum.

- Öffentliche S3-Buckets voller DB-Dumps (Accenture, Verizon usw.).
- Capital One 2019: SSRF brachte eine WAF dazu, ihre IAM-Anmeldedaten preiszugeben → S3.
- Lambda mit s3:* auf *, weil es 'beim ersten Mal funktionierte'.
- Sicherheitsgruppen: 0.0.0.0/0 auf Port 5432.

AWS-Admin: 'Schaff das Root-Konto ab, nutze IAM wo immer nötig.'



Rechteausweitung: DB → OS

Datenbankkonten können zu Shell-Zugriff werden. Drei klassische Züge.

- MS SQL: xp_cmdshell — führt Shell-Befehle als DB-Prozess aus.
- PostgreSQL: COPY ... TO PROGRAM — leitet die Ausgabe an eine Shell weiter.
- MySQL: lade eine UDF-.so-Datei → sys_exec auf dem OS.
- Muster: Jede privilegierte DB-Funktion ist Teil der Angriffsfläche.



Sicherheitslücken in Data Lakes

Alles landet in S3 — auch Dinge, die du nicht dort ablegen wolltest.

- Freizügiges IAM sickert von Compute-Rollen zu Menschen durch.
- Keine erzwungene Verschlüsselung im Ruhezustand für Bucket-Schlüssel.
- Keine schreibseitige Validierung — jeder kann vergiftete Daten ablegen.
- Erste Frage: WER kann in den Lake schreiben? WER kann lesen?



Ransomware liebt Datenbanken

Die DB ist das, was das Unternehmen zurückbraucht — maximales Druckmittel.

- Verschlüsse die Live-Datenbank.
- Verschlüsse die BACKUPS (sonst stellt der Kunde einfach wieder her).
- Exfiltriere zuerst eine Kopie — erpresse mit Veröffentlichung, selbst wenn sie wiederherstellen.
- Verteidigung: offline, unveränderliche Backups + getestete Wiederherstellungen.

Cybersecurity A&D: WannaCry — der Patch war 59 Tage zuvor verfügbar.



Angriffe auf Anmeldedaten

Standard-Anmeldedaten und geleakte Verbindungsstrings — zwei alte, hartnäckige Fehler.

- Redis: standardmäßig jahrelang ohne Passwort.
- Elastic: keine Authentifizierung in der kostenlosen Stufe — ebenfalls jahrelang.
- Connection strings (postgres://user:pass@host) committed to Git.
- Pre-Commit-Secret-Scanning — diese Woche beheben.

Cybersecurity A&D: gestohlene Anmeldedaten = bevorzugter Vektor der organisierten Kriminalität.



Lieferkette — Die DB-Bibliotheken, denen du vertraust

Dein DB-Treiber kam von npm/PyPI/Maven. Genau wie der aller anderen.

- Böartige npm-Pakete, die sich als ORMs / Treiber ausgeben.
- Trojanisierte Docker-Hub-Images für beliebte Datenbanken.
- Kompromittierte PostgreSQL-Erweiterungen / MySQL-Plugins.
- Erweiterungen laufen mit DB-Rechten — meist mit vielen.

Shostack: 'Bedrohungen häufen sich tendenziell an Vertrauensgrenzen.'



RAG / LLM-Datenvergiftung

Stored XSS für das KI-Zeitalter. Schreibe in die Wissensbasis → das LLM zitiert es.

- Vergifteter Support-Artikel: 'Zum Zurücksetzen senden Sie Ihr Passwort an...'
- Vergiftetes internes Dokument: 'Unternehmensrichtlinie ist die Überweisung an...'
- Prompt-Injection eingebettet in abgerufene Dokumente.
- Verteidigung: strikte Schreib-ACLs, Herkunfts-Tags, Mensch im Prozess.

OWASP-2025-Stored-XSS-Prinzip — ersetze 'admin' durch 'LLM'.



Insider-Bedrohungen & laterale Bewegung von DB zu DB

Datenbanken sind keine Endpunkte. Sie sind Knoten in einem Graphen.

- Überberechtigte Dienstkonten. Engineers mit Lesezugriff auf die Produktion.
- DBAs ohne Audit-Trail.
- Oracle DB_LINK, SQL Server Linked Servers, Postgres FDW.
- Gespeicherte Anmeldedaten → kompromittiere A, springe zu B, springe zu C.

Web App Hacker's Handbook: schlecht geschützte Audit-Logs = eine Goldgrube.



Exponierung von Backups & Snapshots

Dieselben Daten wie in der Produktion, halb so viel Sorgfalt.

- Unverschlüsselte DB-Backups auf Freigaben, lesbar für Domain Users.
- Cloud-Snapshots kontoübergreifend geteilt, nie wieder zurückgenommen.
- Externe Backups bei Anbietern, deren Sicherheitslage du nie geprüft hast.
- Backups sind der letzte Ausweg. Wende identify-authN-authZ-audit an.

Schneier: identifizieren, authentifizieren, autorisieren, auditieren — dasselbe Modell.

Deine mentale Landkarte

- ✓ Eine Datenbank = organisierter Speicher + eine Möglichkeit, Fragen zu stellen.
- ✓ Zehn Familien, jede mit ihrer eigenen Falle.
- ✓ Cloud = geteilte Verantwortung. Du verantwortest Konfiguration & Daten.
- ✓ Markdown ist eine Datenbank für LLMs. RAG-Vergiftung ist real.
- ✓ SQLi wurde behoben. Die Angriffsfläche hat sich vergrößert.

Was du nächste Woche tun solltest

1. Installiere PostgreSQL lokal. Schreibe ein SELECT.
2. Mache die PortSwigger-SQL-Injection-Labs (kostenlos).
3. Durchsuche Shodan / Censys — nur Systeme, die dir gehören.
4. Lies einen Breach-Bericht — Capital One, Equifax, MOVEit.
5. Finde heraus, welche Datenbanken DEINE Organisation tatsächlich betreibt.

Lesenswerte Ressourcen zum Speichern

◆ **PortSwigger Web Security Academy**

portswigger.net/web-security

◆ **SQLBolt**

sqlbolt.com

◆ **Use The Index, Luke**

use-the-index-luke.com

◆ **DB-Engines Ranking**

db-engines.com

◆ **OWASP Top 10**

owasp.org/Top10/

**„Datenbanken sind dort,
wo das Geld ist. Buchstäblich.“**

~Larry Ellison (Vermutlich)

Jetzt weißt du, wo du suchen musst.

Quellen — Zitierte Werke

- Beaulieu, A. — Learning SQL, 2nd Edition (O'Reilly)
- Clarke, J. et al. — SQL Injection Attacks and Defense (Syngress)
- Crowther, P. — Concise Guide to Databases (Springer, 2013)
- DeBarros, A. — Practical SQL (No Starch Press)
- Diogenes & Ozkaya — Cybersecurity Attack and Defense Strategies (Packt)
- Janca, T. — Alice and Bob Learn Application Security (Wiley)
- Mishra & Singh — PostgreSQL Development Essentials (Packt)
- MongoDB Cookbook (Packt) • 10gen — Top 5 NoSQL Considerations
- OWASP Top 10 — Ausgabe 2025
- Sharma, M. — Cosmos DB for MongoDB Developers (Apress, 2018)
- Schneier, B. — Secrets and Lies (Wiley)
- Stuttard & Pinto — Web Application Hacker's Handbook, 2nd Ed.
- Shostack, A. — Threat Modeling: Designing for Security (Wiley)
- AWS Administration: The Definitive Guide (Packt)
- Fundamentals of Azure, 2nd Edition (Microsoft Press)

Fragerrunde*

A == Abwesende Blicke